EXPERIMENT 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: Shrivarthini | Register Number: 241401100

Title

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case – Enterprise Vehicle Service & Repair Request Management System.

Aim

To study the limitations of low-code platforms when they are applied to complex enterprise-level applications, using the Vehicle Service & Repair Request Management application built in Zoho Creator as the starting point, and to understand when pro-code extensions or a hybrid architecture become necessary.

Objectives

- To describe how an enterprise-scale Vehicle Service & Repair Request Management System is structured, from users to analytics.
- To map the standard low-code capabilities (forms, reports, workflows, data model, roles) to the operations of the system.
- To investigate the engineering ceilings, bottlenecks and scaling limits of low-code at enterprise scale through eight core limitations.
- To present a three-tier hybrid architecture (low-code application, pro-code extensions, enterprise systems) as the resolution.
- To judge when the choice between low-code, pro-code and hybrid development should be made.

Software / Tools Required

- Zoho Creator (web-based low-code application development platform)
- Vehicle Service & Repair Request Management application (Service_Request_Form and Service_Request_Report) created in Zoho Creator
- Web browser with an active internet connection

Introduction

Low-code platforms allow applications to be developed mainly through visual tools instead of large amounts of traditional program code. Applications are assembled from predefined components such as forms, reports and dashboards, and behaviour is defined through workflows, rules and configuration, with a small amount of scripting for specific needs. This reduces development time and effort and lets a wider range of people take part in building business applications.

Low-code platforms suit standard business applications. As the requirements of an enterprise grow in users, data volume, process complexity, integrations, security and customisation, the built-in capabilities of a platform may no longer be sufficient on their own. This case study examines such limitations by scaling up the Vehicle Service & Repair Request Management application from a single service center to an enterprise-wide service network.

241401100

Case Study: Enterprise Vehicle Service & Repair Request Management System

The Vehicle Service & Repair Request Management application collects customer service requests through Service_Request_Form, lists them in Service_Request_Report, and presents KPI cards and charts on a dashboard. Access is controlled through the role hierarchy Service Center Manager → Service Advisor → Mechanic and a Read Only data-sharing rule. Suppose an organisation must now run a network of many service centers across a region, handling a very large number of requests throughout the year. The system would then serve the following users and functions:

Category	Items
Users	Customers (Individual, Fleet and Corporate), Service Advisors, Mechanics, Service Center Managers and administrators
Data and operations	Customer and vehicle records, service requests, vehicle type and service type selection, appointment scheduling and service history
Supporting functions	Confirmation and status notifications, payments and invoicing, and spare parts inventory
Enterprise integration and insight	CRM integration, analytics and reporting

Figure 13.1 shows the architecture of this system on a low-code platform, arranged as six groups connected from left to right.

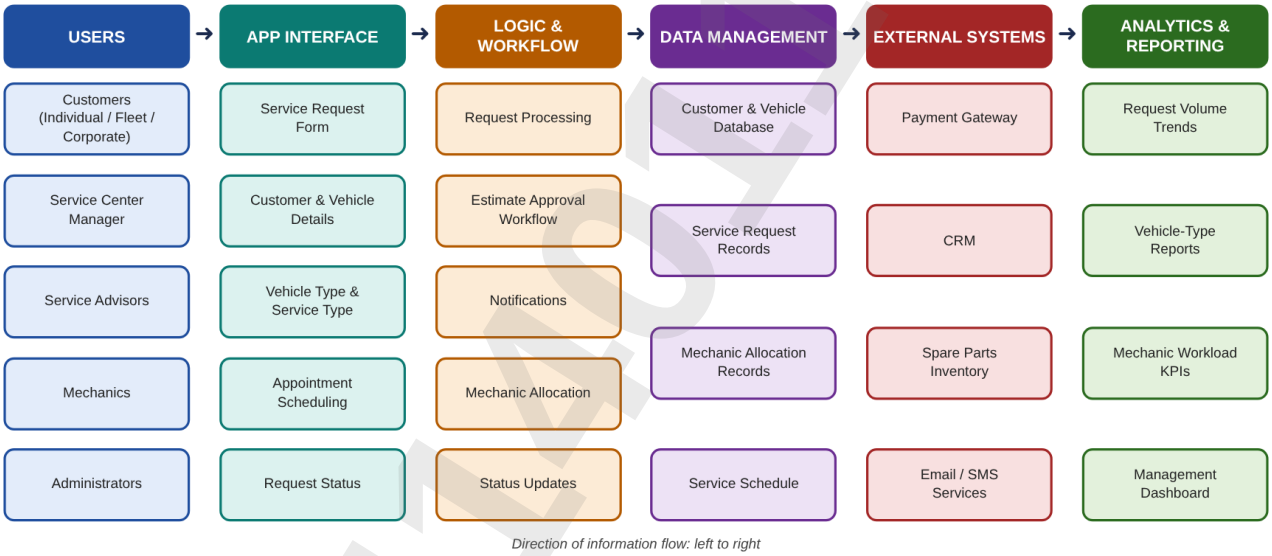


Figure 13.1: Six-Column Architecture of the Enterprise Vehicle Service & Repair Request Management System

- **Users:** customers, service center manager, service advisors, mechanics and administrators, who interact with the system.
- **App Interface:** the Service Request form, customer and vehicle details, vehicle type and service type selection, appointment scheduling and request status.
- **Logic & Workflow:** request processing, estimate approval workflow, notifications, mechanic allocation and status updates, which move a request through the process.
- **Data Management:** the customer and vehicle database, service request records, mechanic allocation records and service schedule, in which the information is stored.
- **External Systems:** the payment gateway, CRM, spare parts inventory and email / SMS services with which the application must exchange information.
- **Analytics & Reporting:** request volume trends, vehicle-type reports, mechanic workload KPIs and a management dashboard for decision makers.

The groups interact in sequence. Users enter and view information through the application; the logic and workflow layer processes it; the data layer stores and supplies it; external systems provide or receive related information such as payments, customer relationships and spare parts stock; and the analytics layer reports on the results.

How a Low-Code Platform Can Support the System

A large part of the system is standard application functionality that low-code platforms are designed to handle:

Capability	Use in the Vehicle Service & Repair Request System
Forms	Service_Request_Form captures customer details, vehicle number, vehicle type, service type, preferred date and issue description.
Reports	Service_Request_Report and its display formats (list, Kanban) show requests by vehicle type, service type and status.
Workflows	Move a request through processing, estimate approval, mechanic allocation and status updates.
Business process automation	Rules and automatic actions, such as updating a status or sending a confirmation when a step is completed.
Data management	Stores customer, vehicle and request records and performs standard create, read, update and delete (CRUD) operations.
User interfaces	Web and mobile screens for customers, advisors and mechanics, with a dashboard of KPI cards and charts.
Roles and sharing	Role hierarchy (Service Center Manager → Service Advisor → Mechanic) and a Read Only data-sharing rule control who sees which records.
Basic integrations	Connect to common services through built-in connectors or standard interfaces.

The Eight Core Limitations at Enterprise Scale

As the system grows into a complex enterprise platform, the following limitations become significant. Figure 13.2 summarises how these limitations arise: a basic low-code application grows into growing enterprise requirements and then enterprise-scale challenges, which lead to the eight key limitations.

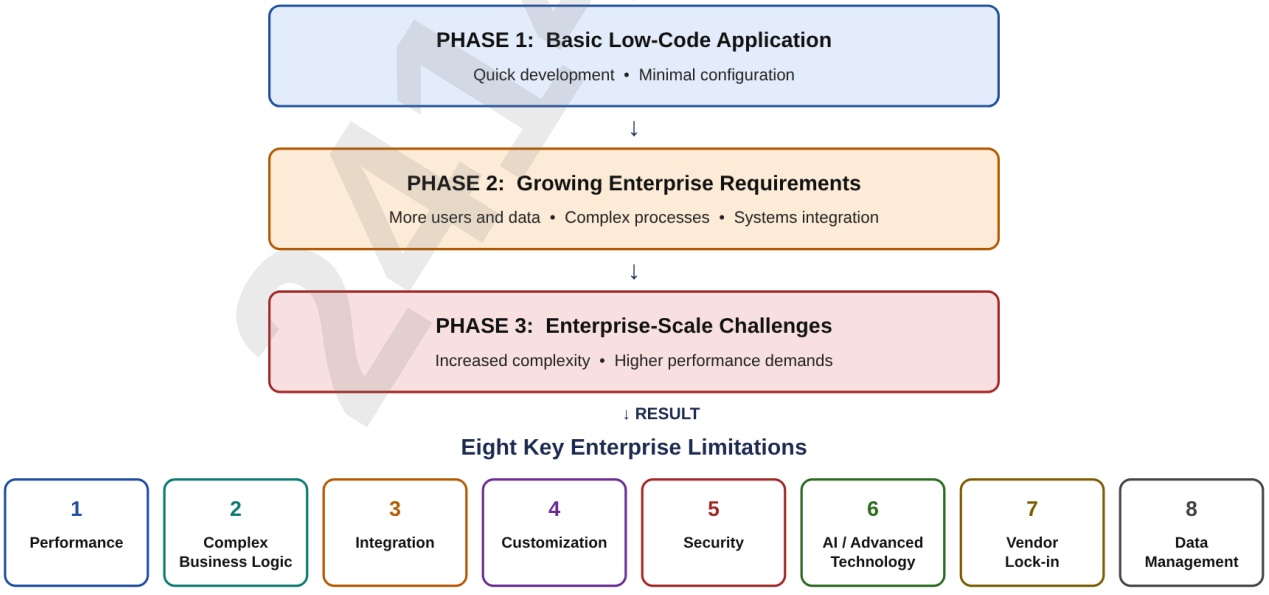


Figure 13.2: Limitations Flowgrid – Limitations of Low-Code Platforms at Enterprise Scale

241401100

1. Performance and Scalability

When service requests, customers and concurrent users grow very large, for example thousands of requests arriving at the same moment during a seasonal rush or a service campaign, performance may be affected. Complex queries and reports over large amounts of data respond slowly, and the platform may impose limits on how far an application can scale. Because the underlying infrastructure is managed by the platform, the options for tuning performance are limited.

2. Complex Business Logic

Highly specialised rules, such as intricate multi-level estimate approval chains, optimised allocation of mechanics and service bays under skill, capacity and clash constraints, and re-ordering of the job queue by priority, are difficult to express with visual workflows and standard scripting. Such requirements may need custom development.

3. System Integration

Connecting the application to ERP and CRM systems, payment gateways, legacy workshop systems, spare parts supplier APIs and messaging services can be challenging. Built-in connectors may not cover every system or data format, and complex or real-time integrations may need custom interfaces.

4. Extreme UX Customization

Standard components provide common screens and layouts. Highly specialised interfaces, such as an interactive service-slot picker, a branded customer self-service portal or a kiosk check-in experience at the service center, may exceed what the standard components can provide.

5. Security and Governance

Enterprises require detailed access control, governance, compliance and strong security practices. Role- and permission-based controls, such as the role hierarchy and Read Only sharing rule used here, may need to be supplemented to meet fine-grained, regulatory or organisation-specific requirements for personal, vehicle and payment-related data, and governance of many applications must be managed carefully.

6. AI / ML and Advanced Technology

Specialised artificial-intelligence and machine-learning models, such as predicting repair time or spare parts demand, or recommending services from a vehicle’s history, may require external services or custom code, since a low-code platform may not provide such capabilities directly. In this account, AI assistance was not available for use (Experiment 9).

7. Vendor / Platform Dependency

The application depends on the capabilities, restrictions, pricing and supported integrations of the selected platform. Features may be limited by the plan or edition (for example the trial and upgrade restrictions seen in earlier experiments), and moving the application to another platform later can be difficult.

8. Heavy Data Transformations

Large datasets, complex relationships between records and advanced processing, such as heavy batch imports of vehicle records, de-duplication of customers across service centers, or complex data transformations for reporting, may be difficult to handle within the data model and tools provided by the platform.

241401100

Relevance of each limitation to the case study

Limitation	Relevance to the Vehicle Service & Repair Request System
Performance & Scalability	Growing numbers of customers, vehicles and service requests, and reports over service history.
Complex Business Logic	Estimate approvals, mechanic allocation and capacity rules that become more complex over time.
System Integration	Payment gateway, CRM, spare parts inventory and email / SMS services.
UX Customization	Specialised interfaces for advisors and mechanics and a tailored customer experience.
Security & Governance	Different access needs for customers, advisors, mechanics and the service center manager; protection of personal and payment data.
AI / Advanced Technology	Possible repair-time forecasting and service recommendation built on service request data.
Vendor Dependency	Reliance on the platform for all application, workflow and reporting features, and on its plan limits.
Heavy Data Transformations	Bulk imports, de-duplication and large volumes of service history related to customers and vehicles.

Low-Code vs Pro-Code Requirements

Neither approach is universally better; the comparison below indicates the typical role of each.

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard screens, forms and layouts from predefined components.	Highly specialised interfaces beyond standard components.
Business logic	Visual workflows, rules and simple scripting.	Complex rules and advanced processing that visual tools cannot express.
Data management	Built-in data model with standard CRUD operations.	Very large datasets, complex relationships and advanced data processing.
Integrations	Built-in connectors and standard interfaces.	Legacy, specialised or real-time integrations and systems without connectors.
AI / ML	Basic features offered by the platform, where available.	Custom AI models, ML pipelines and specialised analytics.
Security	Role- and permission-based access control provided by the platform.	Fine-grained access control and compliance beyond platform features.
Scalability	Suitable where the platform handles the expected users and data.	High data volumes, large user numbers and demanding performance.

Resolution Topology: Three-Tier Hybrid Architecture

Organisations can combine low-code development with pro-code extensions. Low-code is used to develop standard application functionality quickly, while custom code, APIs, integrations and specialised processing are used for requirements that exceed the built-in capabilities of the platform. The structure consists of three layers: **Low-Code Application → Pro-Code Extensions → Enterprise Systems**. Figure 13.3 shows this architecture: users access the system at the top, and information flows in both directions between the layers.

241401100

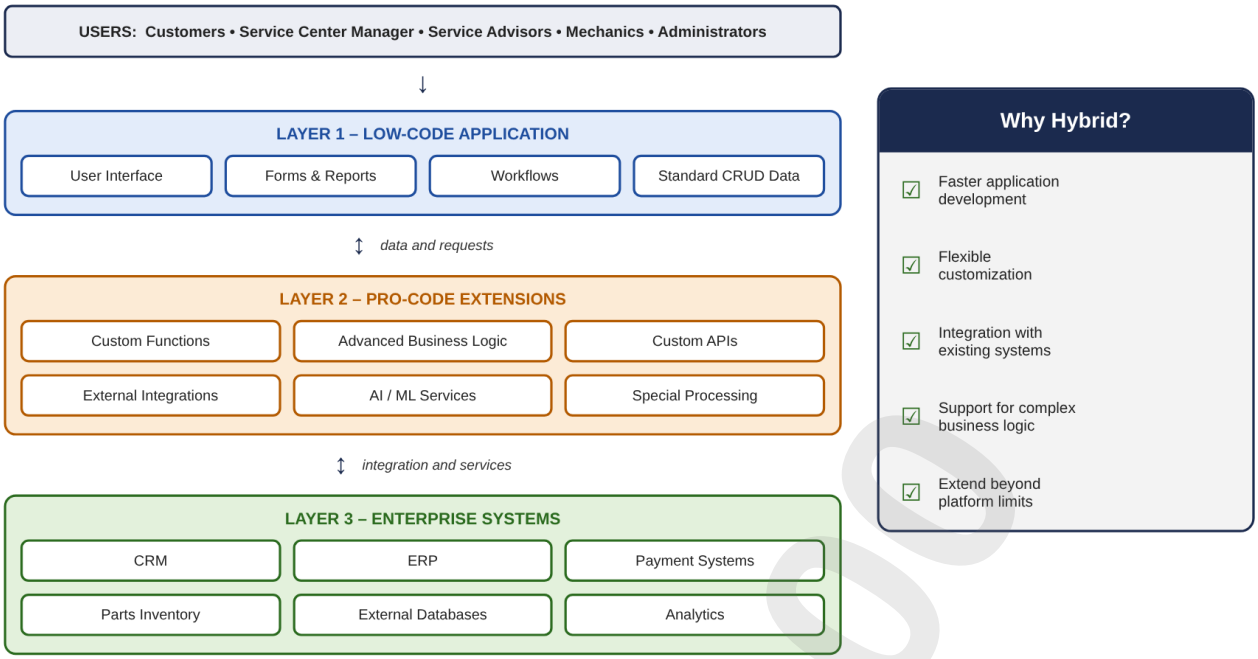


Figure 13.3: Three-Tier Hybrid Low-Code and Pro-Code Enterprise Architecture

Layer	Role in the Vehicle Service & Repair Request System
Layer 1 – Low-Code Application	Rapid development of standard needs: user interface, Service_Request_Form and reports, workflows, role-based access and standard CRUD data management. This is the application built in Zoho Creator.
Layer 2 – Pro-Code Extensions	Custom functions, advanced business logic (for example capacity-aware mechanic allocation), custom APIs, external integrations, AI / ML services (for example repair-time prediction) and special processing such as bulk data transformations.
Layer 3 – Enterprise Systems	Core systems providing data, services and infrastructure: CRM, ERP, payment systems, parts inventory, external databases and analytics platforms.

The “Why Hybrid?” panel of the figure lists the five reasons for a hybrid approach: faster application development, flexible customisation, integration with existing systems, support for complex business logic, and the ability to extend beyond platform limits.

When Pro-Code Extension May Become Necessary

- **Complex business logic:** when rules cannot be represented with standard workflows and rules.
- **Specialised integrations:** when the application must connect to legacy, ERP or other systems without suitable connectors.
- **Custom APIs:** when services or interfaces must be exposed or consumed in a way the platform does not support directly.
- **Advanced AI / ML:** when intelligent features need custom models or ML pipelines.
- **High-performance processing:** when large batches or real-time processing exceed what the platform handles efficiently.
- **Specialised user interfaces:** when the required experience goes beyond standard components.
- **Complex enterprise data processing:** when large datasets and complex relationships need advanced data handling.
- **Requirements beyond platform capabilities:** when needed functionality is simply not offered by the platform.

241401100

Case Study Analysis

For the Vehicle Service & Repair Request System, a low-code platform can be used effectively to develop the standard parts rapidly: the service request form, reports on requests, workflows for processing and approval, notifications, role-based access and dashboards with KPI cards and charts. These correspond to what low-code platforms are designed to provide, and are what the preceding experiments built.

As the organisation grows, requirements become more demanding. Large numbers of customers and service requests raise performance and data-management concerns; integration with payment, CRM and parts inventory systems raises integration concerns; differing access needs and the handling of personal data raise security and governance concerns; and advanced features such as repair-time forecasting need external services. In such cases extensions, integrations or custom development are needed in addition to the low-code application. Low-code is therefore useful for rapidly developing standard business applications, whereas increasingly complex enterprise requirements call for a hybrid approach, and the choice should be based on the requirements of the application.

Result

The case study demonstrated the major limitations that arise when low-code platforms are applied to complex enterprise requirements: performance and scalability, complex business logic, system integration, extreme UX customisation, security and governance, AI / ML and advanced technology, vendor or platform dependency, and heavy data transformations. It also showed how a three-tier hybrid architecture of a low-code application, pro-code extensions and enterprise systems can address requirements beyond standard low-code capabilities.

Conclusion

Low-code platforms offer speed and simplicity for standard business applications, but their limitations become more significant as enterprise complexity increases. The trade-off is clear: low-code gives fast delivery, lower cost and wider participation in development, in exchange for ceilings in scale, logic, integration, customisation, governance and platform independence. For the Vehicle Service & Repair Request Management System, the standard functionality can be built quickly with low-code, while demanding requirements in logic, integration, security, scale and advanced technology need pro-code extensions connected to core enterprise systems. A hybrid architecture keeps the speed of low-code for the standard layer and adds the flexibility of custom code where it is needed, which is why it is regarded as the practical standard for scalable enterprise platforms. It is therefore important to understand the limitations of a platform before selecting a development approach, so that the chosen approach matches the requirements of the enterprise application.

241401100